

**M A S A R Y K
U N I V E R S I T Y**

FACULTY OF INFORMATICS

Implementation of MCP Server for Red Hat OpenShift AI Workbenches

Bachelor's Thesis

ADAM DAVID MALÝ

Brno, Fall 2025

**M A S A R Y K
U N I V E R S I T Y**

FACULTY OF INFORMATICS

Implementation of MCP Server for Red Hat OpenShift AI Workbenches

Bachelor's Thesis

ADAM DAVID MALÝ

Advisor: Honza + Martin

?

Brno, Fall 2025

MUNI
FI

Declaration

Hereby I declare that this paper is my original authorial work, which I have worked out on my own. All sources, references, and literature used or excerpted during elaboration of this work are properly cited and listed in complete reference to the due source.

Adam David Malý

Advisor: Honza + Martin

Abstract

This thesis presents MCP server implementation for RHOAI workbenches.

Keywords

MCP, RHOAI, Workbench, Server, Implementation

Contents

1	Introduction	1
2	Background: Artificial Intelligence and the Model Context Protocol	3
2.1	Large Language Models and AI Assistants	3
2.2	AI Agents and Tool Use	4
2.3	The Model Context Protocol	5
2.3.1	Protocol Overview	5
2.3.2	MCP Components	5
2.3.3	Client Capabilities	6
2.3.4	Server Capabilities	6
2.3.5	Transport Mechanisms	7
3	Background: Red Hat OpenShift AI and Workbenches	9
3.1	Kubernetes and OpenShift	9
3.1.1	Kubernetes Architecture	9
3.1.2	Red Hat OpenShift	11
3.2	Kubeflow, Open Data Hub, and Red Hat OpenShift AI .	12
3.2.1	Kubeflow	12
3.2.2	Red Hat OpenShift AI	13
3.2.3	Open Data Hub	13
3.2.4	Hardware Profiles	14
3.3	Kubeflow Notebooks and Workbenches	14
3.3.1	The Kubeflow Notebook CRD	14
3.3.2	RHOAI Workbenches	15
3.3.3	Notebook Images	15
3.3.4	The Current Management Workflow	16
3.4	MCP Tooling in RHOAI	16
4	Analysis and Design	17
4.1	Motivation and Goals	17
4.2	Requirements	17
4.3	Architecture	17
4.4	Technology Choices	17
5	Implementation	18

5.1	Project Structure	18
5.2	MCP Tools	18
5.2.1	Workbench Management	18
5.2.2	Hardware Profiles	18
5.2.3	Notebook Images	18
5.2.4	Storage and Namespaces	18
5.2.5	Resource Consumption	18
5.3	MCP Resources and Prompts	18
5.4	Security	18
5.5	Code Quality	18
5.6	Evaluation	18
6	Conclusion	19
6.1	Future Work: Notebooks 2.0	19
	Bibliography	20

List of Tables

List of Figures

1 Introduction

Large language models (LLMs) have rapidly evolved in recent years from conversational assistants into capable agents that can reason, plan, and take actions in the real world. This shift has been made possible by the ability to give LLMs access to external *tools* — functions that let a model query a database, call an API, or manage infrastructure, all in response to a natural language request. As the need for such integrations grows, a key challenge emerges: how do different AI hosts and models connect to the ever-expanding set of tools and data sources in a consistent way?

Anthropic introduced the Model Context Protocol (MCP) in November 2024 [1] to address exactly this problem. MCP is an open standard that defines how AI applications connect to external tools and data sources through a client-server architecture. Instead of each AI host maintaining its own integrations, any MCP-compatible host (Cursor, Claude Desktop, or a custom agent) can connect to any MCP server and immediately gain access to the capabilities it exposes. This has already led to a growing ecosystem of MCP servers for services ranging from databases and file systems to cloud APIs and developer tools.

Red Hat OpenShift AI (RHOAI) is an enterprise machine learning platform built on top of Red Hat OpenShift (which is based on Kubernetes). One of its core features are workbenches — containerised JupyterLab environments that data scientists and engineers use for model training and experimentation. Managing clusters with many workbench instances can be a tedious and error-prone task, currently requiring navigation of the RHOAI web UI or direct manipulation of Kubernetes resources. The goal of this thesis is to eliminate that problem by allowing users to manage their workbenches entirely through natural language — simply describing what they want to an AI agent, which then carries out the necessary operations on their behalf.

This thesis presents the design and implementation of an MCP server for RHOAI workbenches, written in Go using the official MCP Go SDK [7]. The server exposes a comprehensive set of tools covering the full lifecycle of workbench management: listing, creating, updating, and deleting workbenches; managing notebook images and hardware profiles; handling persistent volume claims; querying namespaces

and pods; and monitoring resource consumption at the workbench, namespace, user, and cluster level. A safety-oriented read-only mode is provided by default, with write operations gated behind an explicit environment variable. The server also exposes MCP resources and a guided prompt to assist agents in constructing correct workbench creation requests. Quality is validated through unit tests and automated evaluation using the Promptfoo framework [10], which measures tool selection accuracy, parameter extraction correctness, and execution success rate.

2 Background: Artificial Intelligence and the Model Context Protocol

This chapter introduces large language models and the agentic paradigm of tool use, then describes the Model Context Protocol in detail.

2.1 Large Language Models and AI Assistants

At the core of modern AI assistants are *large language models* (LLMs) — neural networks trained to predict and generate text by learning statistical patterns from vast amounts of data. The dominant architectural foundation for LLMs is the *Transformer*, introduced by Vaswani et al. in 2017 [26]. The Transformer’s self-attention mechanism allows the model to relate every token in a sequence to every other token, enabling it to capture long-range dependencies in text far more effectively than earlier recurrent architectures. Another benefit of the Transformer is that it is parallelizable, making it possible to train large models on extensive datasets in a reasonable time.

Modern LLMs are trained in two main stages. In the first stage, *pre-training*, the model is exposed to hundreds of billions of tokens of text and learns to predict the next token in a sequence. This process, while deceptively simple, causes the model to internalise broad world knowledge, reasoning patterns, and linguistic structure. In the second stage, the pre-trained model is fine-tuned to behave as a helpful assistant, typically through a combination of supervised instruction fine-tuning and reinforcement learning from human feedback (RLHF) [9]. The result is a model that responds to natural-language instructions in a coherent and useful way.

The products of this pipeline — systems such as OpenAI’s ChatGPT, Anthropic’s Claude, and Google’s Gemini — are commonly referred to as *AI assistants*. An AI assistant takes a user’s message as input and produces a textual response. It is fundamentally *stateless* and *passive*: it answers questions, drafts documents, and explains concepts, but it does not take actions in the world, does not maintain persistent memory across sessions by default, and cannot access information beyond its training data or the content of the current conversation. This

passivity is the key limitation that the agentic paradigm, described in the next section, overcomes.

2.2 AI Agents and Tool Use

Despite their impressive capabilities, pure language models are isolated from the external world — their knowledge does not extend beyond their training data, they cannot query other systems, and they cannot take actions. The concept of an *AI agent* overcomes these limitations by equipping a language model with *tools* — callable functions that the model can invoke to interact with external systems, retrieve fresh information, or modify state. Agents can also be given *memory* — the ability to retain information across steps or sessions. This can range from simply keeping the full conversation history in the model’s context window, to reading and writing records in an external database via dedicated memory tools.

The key mechanism enabling this is *tool calling* (also referred to as *function calling*). When a model is given a set of tool definitions alongside a user request, it can — instead of producing a plain text response — output a structured call to one of those tools, specifying the tool name and the required arguments. The host application intercepts this call, executes the corresponding function, and feeds the result back into the model’s context, after which the model continues its reasoning. This loop can repeat multiple times within a single interaction, allowing the model to chain tool calls to complete complex, multi-step tasks.

Yao et al. formalised this pattern in *ReAct* [27], which unifies *reasoning* and *acting* into a single loop: the model first produces a chain-of-thought reasoning trace — thinking through what it knows and what it needs — and then selects and executes an action (a tool call). The output returned by the tool becomes part of the context for the next reasoning step. This structured loop gives agents a degree of *autonomy*: given a high-level goal, an agent can decompose it into sub-tasks, select appropriate tools, and adapt its behaviour based on the results it receives.

The practical impact of tool use is significant. An agent equipped with the right tools can list resources in a Kubernetes cluster, query

2. BACKGROUND: ARTIFICIAL INTELLIGENCE AND THE MODEL CONTEXT PROTOCOL

resource consumption metrics, and delete workbench instances — all in response to a single natural-language instruction such as “delete workbenches running for more than a week”. The Model Context Protocol provides a standardised way of exposing those capabilities to any AI host.

2.3 The Model Context Protocol

2.3.1 Protocol Overview

The Model Context Protocol (MCP) is an open standard introduced by Anthropic in November 2024 [1] that defines how AI applications connect to external tools and data sources. Its core motivation is the *M×N integration problem*: without a shared standard, connecting M AI hosts to N external services requires up to $M \times N$ custom integrations, each with its own authentication, serialisation, and error-handling logic. MCP reduces this to $M + N$ by providing a single protocol that both sides implement once.

The protocol is *transport-agnostic* and message-based, built on top of JSON-RPC 2.0 [3]. Official SDKs are available for Python, TypeScript, Go, and other languages [7].

2.3.2 MCP Components

The MCP architecture defines three main components: *hosts*, *clients*, and *servers*.

A **host** is the AI application that the end user interacts with directly — for example Claude Desktop, Cursor, or a custom agent application. The host is responsible for managing the user interaction, sending messages to the LLM, orchestrating tool calls, and processing the output of the LLM.

A **client** is a component that lives inside the host and manages exactly one connection to one MCP server. A single host can run multiple clients simultaneously, each connected to a different server, giving the agent access to multiple sets of tools at once. The client speaks the MCP protocol on behalf of the host — it sends requests to the server, receives responses and notifications, and forwards results back to the LLM context. Beyond handling requests, clients can also expose

2. BACKGROUND: ARTIFICIAL INTELLIGENCE AND THE MODEL CONTEXT PROTOCOL

their own capabilities to servers, enabling server-initiated interactions described in Section 2.3.3.

A **server** is a lightweight process that exposes capabilities (tools, resources, and prompts) to clients [2]. A server typically wraps one specific external system — a database, a cloud API, a file system, or in the case of this thesis, Red Hat OpenShift AI workbenches. It contains the actual implementation of each capability. When a tool is invoked, the server executes the corresponding logic and returns the result. The server does not communicate with the LLM directly — it only communicates with the client that is connected to it. This separation of roles means that server authors do not need to know anything about which LLM or host will use their tools and other capabilities. The server-exposed primitives are described in Section 2.3.4.

2.3.3 Client Capabilities

While servers expose tools, resources, and prompts, clients can expose their own capabilities back to servers, allowing servers to initiate certain interactions rather than only responding to them. MCP defines three such client capabilities [2].

Sampling lets a server ask the client to generate an LLM completion on its behalf. This enables servers to perform agentic sub-tasks — such as summarising a result or deciding a next step — without needing direct access to any language model.

Roots allow a server to query the client about the filesystem or URI boundaries it is permitted to operate in, so that servers respect the scope defined by the host environment.

Elicitation lets a server request additional information from the user through the host, for example asking for a confirmation or a missing parameter before proceeding with an operation.

2.3.4 Server Capabilities

MCP servers expose capabilities to clients through three primitive types: tools, resources, and prompts [2].

Tools are the primary mechanism for giving an AI agent the ability to act. Each tool has a name, a human-readable description, and a JSON Schema that describes its input parameters. When the LLM de-

2. BACKGROUND: ARTIFICIAL INTELLIGENCE AND THE MODEL CONTEXT PROTOCOL

cides that a tool is needed to fulfil a request, it emits a structured tool call containing the tool name and arguments. The host executes the call via the client and returns the result to the model’s context. Tools are *model-initiated* — it is the LLM that decides when and how to invoke them. This thesis implements tools for the full workbench management lifecycle: listing, creating, updating, and deleting workbenches, notebook images, hardware profiles, persistent volume claims, and namespaces, as well as tools for monitoring resource consumption.

Resources are URI-addressed data that the host can read and inject into the model’s context. Unlike tools, resources are *host-initiated* — it is the host or the user that decides which resources to include, not the model. While conceptually similar to Retrieval-Augmented Generation (RAG) [6], where relevant documents are retrieved and injected into the prompt, MCP resources differ in that they are explicitly declared by the server and selected by the host or user, rather than retrieved automatically through a similarity search. This thesis exposes two resources: an *Image Catalog*, which lists all notebook images available in the cluster with their versions and Python dependencies; and a *Default Hardware Profile*, which describes the default CPU and memory allocation for new workbenches. An agent can read these resources before calling the Create Workbench tool to ensure it selects a valid image and reasonable resource limits.

Prompts are pre-defined, parameterisable message templates that guide the model’s behaviour for a specific task. A prompt can be invoked by the user or the host and expands into one or more messages that are injected into the conversation. This thesis provides a *Create Workbench* prompt that instructs the agent to first consult the Image Catalog resource to select a suitable image and then invoke the Create Workbench tool — a pattern that combines all three MCP server primitives in a single guided workflow.

2.3.5 Transport Mechanisms

MCP is transport-agnostic (the protocol defines the structure of messages but not how they are physically carried between endpoints) and defines two standard transport mechanisms [2] — these are the transports supported by the official SDKs [7] and used by the ecosystem in practice. All messages follow the JSON-RPC 2.0 format [3], a

2. BACKGROUND: ARTIFICIAL INTELLIGENCE AND THE MODEL CONTEXT PROTOCOL

lightweight remote procedure call protocol in which each request and response is a single JSON object identified by a method name and a unique request identifier.

Standard I/O (stdio) is the simplest transport. The host launches the MCP server as a child process and communicates with it over the process's standard input and standard output streams. Each JSON-RPC message is a newline-delimited JSON object. This transport requires no network configuration, has no authentication overhead, and is inherently scoped to the local machine, making it well-suited for locally-installed tools. The server implemented in this thesis uses the stdio transport, initialised in the entry point:

```
server.Run(context.Background(), &mcp.  
    StdioTransport{})
```

HTTP with Server-Sent Events (SSE) is the remote transport. The client sends requests to the server via HTTP POST, and the server delivers responses and unsolicited notifications back via an SSE stream. This allows MCP servers to be deployed as network services, enabling multi-user scenarios and remote infrastructure management. A newer *Streamable HTTP* transport, which consolidates both directions into a single HTTP endpoint, is also defined in the specification as the recommended transport for new remote deployments [2].

3 Background: Red Hat OpenShift AI and Workbenches

This chapter describes the fundamental concepts of Red Hat OpenShift AI and workbenches. These concepts are the foundation for the MCP server implementation and architecture choices. The chapters follow the layered architecture from bottom to top: first OpenShift (based on Kubernetes) as the infrastructure layer, then Red Hat OpenShift AI (based on KubeFlow) as the machine-learning platform, and finally workbenches (based on KubeFlow notebooks) as the user-facing JupyterLab environments.

3.1 Kubernetes and OpenShift

3.1.1 Kubernetes Architecture

Kubernetes is an open-source platform for automating the deployment, scaling, and management of containerised applications [20]. It was originally designed by Google and is now maintained by the Cloud Native Computing Foundation (CNCF). Kubernetes provides a declarative configuration model: users describe the *desired state* of their applications in YAML or JSON manifests, and the platform’s control loop continuously reconciles the actual state of the cluster to match. This section introduces the core Kubernetes architectural concepts that are relevant to the MCP server implemented in this thesis.

Nodes and the control plane. A Kubernetes cluster consists of a *control plane* and a set of worker machines called *nodes*. The control plane runs the API server, scheduler, and controller manager. They accept user requests, decide where workloads should run, and ensure the desired state is maintained. Each worker node runs a *kubelet* agent that manages the *pods* (the smallest deployable units, described below) scheduled on that node and reports status back to the control plane [20].

Pods. The smallest deployable unit in Kubernetes is a *pod* — a group of one or more containers that share the same network namespace and storage [24]. In practice, most pods contain a single application container. Every workbench in Red Hat OpenShift AI ultimately runs as a pod, and the MCP server lists pods to retrieve workbench uptime.

Namespaces. Namespaces provide a mechanism for isolating groups of resources within a single cluster [22]. In the context of Red Hat OpenShift AI, each data-science project maps to a Kubernetes namespace. Almost every operation exposed by the MCP server — listing workbenches or pods, creating storage, querying resource consumption — is scoped to a namespace.

Persistent Volumes and Persistent Volume Claims. Containers are ephemeral by default: any data written inside a container is lost when the pod is terminated. Kubernetes addresses this with a two-layer storage abstraction [23]. A *PersistentVolume* (PV) represents a piece of storage provisioned in the cluster. A *PersistentVolumeClaim* (PVC) is a user’s request for storage; when a PVC is created, Kubernetes binds it to a suitable PV. Workbenches mount a PVC to persist notebooks and data across restarts, and the MCP server provides tools to create, resize, and delete PVCs.

Labels and annotations. Kubernetes objects carry two kinds of key-value metadata [21]. *Labels* are used for selection and grouping — for example, querying all pods that belong to a particular workbench. *Annotations* store non-identifying metadata such as display names, image version hashes, or hardware profile references. The MCP server reads and writes annotations extensively: workbench status, the selected notebook image, and the hardware profile are all stored as annotations on the underlying Kubernetes object.

StatefulSets. A *StatefulSet* manages a set of pods with stable network identities and persistent storage [25]. Unlike a Deployment, which treats all pods as interchangeable, a StatefulSet guarantees that each pod retains its identity and its associated PVC across rescheduling. Workbenches are backed by StatefulSets, which ensures that a

user’s notebook environment is preserved even if the underlying node changes.

Custom Resource Definitions. One of Kubernetes’ most powerful extensibility features is the ability to define *Custom Resource Definitions* (CRDs) [19]. A CRD registers a new resource type with the Kubernetes API server, allowing users and operators to manage domain-specific objects — such as machine learning notebooks — with the same tools and workflows used for built-in resources like pods or services. The KubeFlow project defines a Notebook CRD (API group `kubeflow.org/v1`) that represents a single notebook environment; this is the resource that the MCP server creates, updates, and deletes when managing workbenches.

3.1.2 Red Hat OpenShift

Red Hat OpenShift is a commercial Kubernetes distribution developed by Red Hat [13]. It packages upstream Kubernetes with a curated set of additional components and wraps them in enterprise support, certified operators, and a hardened security model. While any standard Kubernetes cluster can run containerised workloads, OpenShift adds several features that are particularly relevant to the platform stack discussed in this thesis.

Projects. OpenShift introduces the concept of a *project*, which is a thin wrapper around a Kubernetes namespace that adds default role bindings, network policies, and resource quotas [17]. In practice, every OpenShift project maps one-to-one to a namespace, and the terms are often used interchangeably. Red Hat OpenShift AI uses projects to represent data-science teams; the MCP server operates on these projects through the underlying Kubernetes namespace API.

ImageStreams. Standard Kubernetes references container images by their full registry URL. OpenShift extends this model with *ImageStreams* [12], which are cluster-level objects that track one or more tagged versions of a container image. An ImageStream decouples workloads from specific registry URLs: when a new image version is

pushed, the ImageStream is updated and can automatically trigger redeployments. Red Hat OpenShift AI uses ImageStreams (API group `image.openshift.io/v1`) to manage the catalogue of notebook images available for workbenches. The MCP server queries and creates ImageStreams when listing available images or registering custom notebook images.

Routes and built-in OAuth. OpenShift provides *Routes* as a native mechanism for exposing HTTP and HTTPS services to external traffic [11], fulfilling a role similar to Kubernetes Ingress but with tighter integration into the platform. Each workbench is exposed through a Route, which gives it a stable URL. OpenShift also ships a built-in OAuth server that handles user authentication [15]. Workbenches leverage this to ensure that only authenticated users can access their JupyterLab environments.

Operators and the Operator Lifecycle Manager. OpenShift uses the *Operator* pattern extensively to manage complex software on top of the cluster [16]. An Operator is a Kubernetes controller that watches custom resources and reconciles them toward their desired state. The Operator Lifecycle Manager (OLM) handles installation, upgrades, and dependency resolution for Operators. Red Hat OpenShift AI itself is installed as an Operator, which in turn deploys and manages the Kubeflow components, dashboard, and other services that make up the platform.

3.2 Kubeflow, Open Data Hub, and Red Hat OpenShift AI

3.2.1 Kubeflow

Kubeflow is an open-source project that aims to make deployments of machine learning workflows on Kubernetes simple, portable, and scalable [4]. Rather than being a single monolithic application, Kubeflow is a collection of components — each addressing a different stage of the ML lifecycle — that are deployed as Operators and custom resources on a Kubernetes cluster. Key components include Kubeflow Pipelines

for orchestrating multi-step training workflows, KServe for model serving, Katib for hyperparameter tuning, and Kubeflow Notebooks for interactive development environments.

From the perspective of this thesis, the most important Kubeflow component is **Kubeflow Notebooks**, which provides a safe, scalable place for data scientists to write code. The Notebook CRD, its controller, and its relationship to RHOAI workbenches are discussed in detail in Section 3.3.

3.2.2 Red Hat OpenShift AI

Red Hat OpenShift AI (RHOAI) is Red Hat’s commercial machine-learning platform built on top of OpenShift [14]. It is based on Open Data Hub (ODH) [8], a community-driven project that integrates upstream Kubeflow components with OpenShift-specific features. RHOAI is installed on an OpenShift cluster as an Operator and provides a web-based *dashboard* — a graphical interface from which users can create data-science projects, launch and manage workbenches, configure notebook images and hardware profiles, and monitor resource consumption without writing any YAML or using the command line.

3.2.3 Open Data Hub

Open Data Hub (ODH) is the upstream open-source community project from which Red Hat OpenShift AI is derived [8]. ODH provides the same core components — the dashboard, notebook controller, model serving, and pipeline infrastructure — but without the enterprise hardening, certified Operators, or commercial support that RHOAI adds. Many of the custom resource API groups used by the MCP server (such as `opendatahub.io`) originate from ODH and are inherited by RHOAI. The CRDs, labels, and annotations are largely shared between the two platforms. However, the MCP server targets RHOAI specifically because it assumes RHOAI-specific defaults such as the `redhat-ods-applications` namespace for discovering hardware profiles.

3.2.4 Hardware Profiles

A *hardware profile* is an RHOAI-specific custom resource that defines a named set of compute-resource constraints for a workbench. Each profile specifies one or more resource identifiers — typically CPU, memory, and optionally a GPU (referred to as an *accelerator* in RHOAI) — with a default, minimum, and maximum count for each. When a user creates a workbench, they select a hardware profile. Platform then translates the profile’s resource definitions into Kubernetes resource requests and limits on the workbench’s pod. Administrators can create hardware profiles to standardise the compute tiers available to data scientists (for example, “Small” with 2 CPUs and 4 GiB of memory, or “GPU Large” with 8 CPUs, 32 GiB, and one NVIDIA GPU). The MCP server provides tools to list, create, update, and delete hardware profiles and to attach them to workbenches.

3.3 Kubeflow Notebooks and Workbenches

This section describes the topmost layer of the platform stack: the interactive development environments that data scientists use daily. The upstream Kubeflow project calls them *Notebooks*; Red Hat OpenShift AI calls them *workbenches*. Under the hood they are the same Kubernetes resource, but RHOAI adds additional metadata and dashboard integration.

3.3.1 The Kubeflow Notebook CRD

The Kubeflow Notebook Controller defines a Custom Resource Definition called Notebook in the API group `kubeflow.org/v1` [5]. A Notebook resource is structurally similar to a Kubernetes `StatefulSet`: it contains a pod template that specifies the container image, resource requests and limits, volume mounts, and environment variables. When a Notebook resource is created, the Notebook Controller creates the corresponding `StatefulSet`, which then creates a pod running the specified container image — typically a JupyterLab server.

The Notebook Controller also handles lifecycle operations. Setting the annotation `kubeflow-resource-stopped` to a timestamp causes the controller to scale the `StatefulSet` to zero replicas, effectively stop-

ping the notebook without deleting it. Removing the annotation restarts it. This mechanism is how both the RHOAI dashboard and the MCP server start and stop workbenches.

3.3.2 RHOAI Workbenches

In Red Hat OpenShift AI, a *workbench* is a KubeFlow Notebook resource enriched with additional annotations and integrated into the RHOAI dashboard [18]. The RHOAI dashboard is the primary way users interact with workbenches today. When a user creates a workbench through the dashboard, it constructs a Notebook manifest with RHOAI-specific annotations such as:

- `opendatahub.io/image-display-name` — the human-readable name of the selected notebook image,
- `opendatahub.io/hardware-profile-name` — the hardware profile governing CPU, memory, and GPU limits,
- `opendatahub.io/username` — the owner of the workbench,
- `notebooks.opendatahub.io/last-image-selection` — a reference to the selected ImageStream tag for upgrade tracking.

The dashboard then submits this manifest to the Kubernetes API, where the Notebook Controller picks it up and creates the running environment. Each workbench is backed by a PersistentVolumeClaim that stores the user's data, and is exposed to the user via an OpenShift Route with OAuth authentication. The MCP server implemented in this thesis follows the same approach: it constructs Notebook manifests with the same annotations and submits them to the Kubernetes API, effectively replicating what the dashboard does but driven by an AI agent through natural language.

3.3.3 Notebook Images

RHOAI ships a curated set of notebook images, each used for a specific workflow. Examples include images with pre-installed data-science libraries (NumPy, pandas, scikit-learn), PyTorch or TensorFlow for deep learning, and minimal images for lightweight experimentation and testing. Each image is represented as an OpenShift ImageStream (see Section 3.1.2) with one or more version tags. Custom images — for example, an image with a development version of a library not

yet available in the default catalogue — can be registered through the RHOAI dashboard or by creating an ImageStream directly. The MCP server provides tools to list (with additional metadata), create, update, and delete notebook images.

3.3.4 The Current Management Workflow

Today, managing workbenches requires either navigating the RHOAI web dashboard or directly manipulating Kubernetes resources through `oc` or `kubectl`. Both approaches have limitations. The dashboard is user-friendly but manual: creating, stopping, or reconfiguring multiple workbenches is time-consuming. The command-line approach is scriptable but requires deep knowledge of the underlying CRDs, annotations, and resource relationships described in this chapter. The MCP server implemented in this thesis bridges this gap by exposing all workbench management operations as tools that an AI agent can invoke through natural language.

3.4 MCP Tooling in RHOAI

I think this subchapter is not needed – Lets do this subchapter only if there is not enough text in thesis.

4 Analysis and Design

4.1 Motivation and Goals

4.2 Requirements

4.3 Architecture

4.4 Technology Choices

5 Implementation

5.1 Project Structure

5.2 MCP Tools

5.2.1 Workbench Management

5.2.2 Hardware Profiles

5.2.3 Notebook Images

5.2.4 Storage and Namespaces

5.2.5 Resource Consumption

5.3 MCP Resources and Prompts

5.4 Security

5.5 Code Quality

5.6 Evaluation

6 Conclusion

6.1 Future Work: Notebooks 2.0

Bibliography

- [1] Anthropic. *Introducing the Model Context Protocol*. Nov. 25, 2024. URL: <https://www.anthropic.com/news/model-context-protocol> (visited on 03/10/2026).
- [2] Anthropic. *Model Context Protocol Specification*. 2024. URL: <https://modelcontextprotocol.io/specification/latest> (visited on 03/10/2026).
- [3] JSON-RPC Working Group. *JSON-RPC 2.0 Specification*. 2013. URL: <https://www.jsonrpc.org/specification> (visited on 03/10/2026).
- [4] Kubeflow Contributors. *Kubeflow Documentation*. 2024. URL: <https://www.kubeflow.org/docs/> (visited on 03/22/2026).
- [5] Kubeflow Contributors. *Notebook CRD — Kubeflow GitHub Repository*. 2024. URL: https://github.com/kubeflow/kubeflow/blob/master/components/notebook-controller/config/crd/bases/kubeflow.org_notebooks.yaml (visited on 03/22/2026).
- [6] Patrick Lewis et al. “Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks”. In: *Advances in Neural Information Processing Systems*. Vol. 33. 2020, pp. 9459–9474. URL: <https://arxiv.org/abs/2005.11401>.
- [7] Model Context Protocol Contributors. *Model Context Protocol — GitHub Organisation*. 2024. URL: <https://github.com/modelcontextprotocol> (visited on 03/10/2026).
- [8] Open Data Hub Contributors. *Open Data Hub Documentation*. 2024. URL: <https://opendatahub.io/docs/> (visited on 03/22/2026).
- [9] Long Ouyang et al. “Training language models to follow instructions with human feedback”. In: *Advances in Neural Information Processing Systems* 35 (2022), pp. 27730–27744. URL: <https://arxiv.org/abs/2203.02155>.
- [10] Promptfoo Contributors. *Promptfoo: Test and evaluate LLMs*. 2024. URL: <https://www.promptfoo.dev> (visited on 03/10/2026).
- [11] Red Hat. *Configuring Routes — OpenShift Container Platform Documentation*. 2024. URL: <https://docs.openshift.com/container-platform/latest/networking/routes/route-configuration.html> (visited on 03/22/2026).

BIBLIOGRAPHY

- [12] Red Hat. *Managing Image Streams — OpenShift Container Platform Documentation*. 2024. URL: https://docs.openshift.com/container-platform/latest/openshift_images/image-streams-manage.html (visited on 03/22/2026).
- [13] Red Hat. *OpenShift Container Platform Documentation*. 2024. URL: <https://docs.openshift.com/container-platform/latest/welcome/index.html> (visited on 03/22/2026).
- [14] Red Hat. *Red Hat OpenShift AI Documentation*. 2024. URL: https://docs.redhat.com/en/documentation/red_hat_openshift_ai_self-managed/ (visited on 03/22/2026).
- [15] Red Hat. *Understanding Authentication — OpenShift Container Platform Documentation*. 2024. URL: <https://docs.openshift.com/container-platform/latest/authentication/understanding-authentication.html> (visited on 03/22/2026).
- [16] Red Hat. *Understanding Operators — OpenShift Container Platform Documentation*. 2024. URL: <https://docs.openshift.com/container-platform/latest/operators/understanding/olm-what-operators-are.html> (visited on 03/22/2026).
- [17] Red Hat. *Working with Projects — OpenShift Container Platform Documentation*. 2024. URL: <https://docs.openshift.com/container-platform/latest/applications/projects/working-with-projects.html> (visited on 03/22/2026).
- [18] Red Hat. *Working with Workbenches — Red Hat OpenShift AI Documentation*. 2024. URL: https://docs.redhat.com/en/documentation/red_hat_openshift_ai_self-managed/2-latest/html/working_on_data_science_projects/using-project-workbenches_workbenches (visited on 03/22/2026).
- [19] The Kubernetes Authors. *Custom Resources — Kubernetes Documentation*. 2024. URL: <https://kubernetes.io/docs/concepts/extend-kubernetes/api-extension/custom-resources/> (visited on 03/22/2026).
- [20] The Kubernetes Authors. *Kubernetes Documentation*. 2024. URL: <https://kubernetes.io/docs/home/> (visited on 03/22/2026).
- [21] The Kubernetes Authors. *Labels and Selectors — Kubernetes Documentation*. 2024. URL: <https://kubernetes.io/docs/concepts/overview/working-with-objects/labels/> (visited on 03/22/2026).

- [22] The Kubernetes Authors. *Namespaces — Kubernetes Documentation*. 2024. URL: <https://kubernetes.io/docs/concepts/overview/working-with-objects/namespaces/> (visited on 03/22/2026).
- [23] The Kubernetes Authors. *Persistent Volumes — Kubernetes Documentation*. 2024. URL: <https://kubernetes.io/docs/concepts/storage/persistent-volumes/> (visited on 03/22/2026).
- [24] The Kubernetes Authors. *Pods — Kubernetes Documentation*. 2024. URL: <https://kubernetes.io/docs/concepts/workloads/pods/> (visited on 03/22/2026).
- [25] The Kubernetes Authors. *StatefulSets — Kubernetes Documentation*. 2024. URL: <https://kubernetes.io/docs/concepts/workloads/controllers/statefulset/> (visited on 03/22/2026).
- [26] Ashish Vaswani et al. “Attention Is All You Need”. In: *Advances in Neural Information Processing Systems* 30 (2017). URL: <https://arxiv.org/abs/1706.03762>.
- [27] Shunyu Yao et al. “ReAct: Synergizing Reasoning and Acting in Language Models”. In: *International Conference on Learning Representations*. 2023. URL: <https://arxiv.org/abs/2210.03629>.